GlycoTwin — Architecture

Personalized Post-Meal Glucose Digital Twin · investigational research prototype

BROWSER

React 19 · TypeScript · Vite
Pages: Home, My Twin, Forecast, Observations, History, Insights, Settings
Hash router; calls `/api/...`

→

FASTAPI BACKEND (bound to 127.0.0.1 only)

Routers: **auth**, **twins**
HttpOnly session cookie + ownership check on every `/twins/{id}...` route
scrypt password hashing · one DB transaction per operation

→

SQLITE · 11 TABLES

users · sessions · twin_owners · twins · twin_states · parameter_snapshots · forecasts · observations · reconciliations · what_if_runs · data_quality_flags

RESEARCH ENGINE (shared Python package)

Bayesian Models **B** and **C** — closed-form conjugate update, `forecast_exceeds_180` · versioned twin state · explanations · what-if comparisons · data-quality flags.
The live twin and the offline research experiments call the exact same functions — no separate "demo" model.

CLAIM / OWNERSHIP TOOL

`scripts/claim_twin.py` assigns pre-account twins to a real account (dry-run by default). Every twin has exactly one owner; another account's twin returns the identical 404 as a twin that does not exist.

THE FORECAST → OBSERVE → UPDATE LOOP

meal inputs → validation → **forecast + uncertainty** (90% interval, P(max ≥ 180 mg/dL)) → observed outcome (after the 120-min window closes) → **eligibility & leakage safeguards** → **Bayesian parameter update** → new twin version saved → next forecast uses the updated state, and full history is kept

OFFLINE CGMacros RESEARCH PIPELINE (researcher's own machine — never reached by the website)

Meal-event table (t0 = logged meal row, 120-min window, label = max ≥ 180 mg/dL) → leakage safeguards (participant-level folds, sequential evaluation) → Model A (population XGBoost baseline) → Model B vs Model C comparison and reports. **Only aggregate priors and reports ever cross over to the website — participant-level data never does.**